Zarqa University

Faculty of Information Technology

Software Engineering Department

Builder Design Pattern

Subject: Patterns Programming

Presented by:

Saja Fateh, (202402627)

Shahed Mahmoud, (202430155)

Ghofran Al-Shareef, (202430081)

Malak Saleh, (202302115)

Supervised by:

Dr. Issam Jebreen

Contents

Introduction To Builder Design Pattern 2
- **1.1 Overview of Builder design patterns:** 2
- **1.2 Intent of the pattern:** 2
- **1.3 Key idea (step-by-step construction):** 2
- **Problem Analysis & Trade-offs** 3
- **2.1 Issues in object creation:** 3
  - **2.1.1 Telescoping constructors:** 3
  - **2.1.2 Too many parameters:** 3
  - **2.1.3 Inconsistent object state:** 4
- **2.2 Why traditional approaches fail:** 4
- **2.3 Possible design patterns to solve these issues:** 4
  - **2.3.1 Builder Design Pattern** 4
  - **2.3.2 Abstract Factory Design Pattern** 4
- **Pattern Selection & Justification** 5
- **3.1 Why Builder Design Pattern is Needed:** 5
- **3.2 Real-World Analogy:** 5
- **3.3 When to Use the Builder Pattern:** 6

Case Study: Game Character Creation and UML Diagrams 6
- **4.1 Problem in Game Character Creation:** 6
- **4.2 Applying Builder Pattern as a Solution:** 7

Pattern Implementation 8
- **5.1 Code without Builder:** 8
- **5.2 Code with Builder:** 9
- **5.3 Key differences:** 12

Advantages of Builder Pattern 13
- **Advantages:** 13

Disadvantages of Builder Pattern 13
- **Disadvantages:** 13

Conclusion 14
- **8.1 Summary of the pattern:** 14
- **8.2 Why it was chosen:** 14

References 15
- **Books** 15
- **Websites** 15

Introduction To Builder Design Pattern

1.1 Overview of Builder design patterns:

The Builder Design Pattern is a creational design pattern that focuses on constructing complex objects step by step. It is particularly useful when an object requires multiple parameters, especially when some of them are optional.

Instead of creating objects using a single constructor with many parameters, the Builder pattern separates the construction process from the final representation. This allows developers to build objects gradually and more clearly.

The pattern improves code readability, flexibility, and maintainability by providing a structured way to create objects. It also helps avoid common issues such as telescoping constructors and inconsistent object states.

As part of the creational design patterns category, the Builder pattern emphasizes controlled and organized object creation, making it highly suitable for complex systems.

1.2 Intent of the pattern:

The main intent of the Builder Design Pattern is to:

- **Construct complex objects step by step**
- **Allow different representations of the same object**
- **Improve code readability and flexibility**
- **Avoid issues such as telescoping constructors and inconsistent object states**

By using this pattern, the process of object creation becomes more controlled and organized.

1.3 Key idea (step-by-step construction):

The core idea of the Builder Design Pattern is to build an object incrementally by setting its components step by step. Instead of passing all parameters at once through a constructor, the object is constructed through a series of method calls.

This approach allows developers to:

- **Specify only the required attributes**
- **Omit optional parameters easily**
- **Maintain a clear and readable construction process**

Problem Analysis & Trade-offs

2.1 Issues in object creation:

In object-oriented programming, creating complex objects can become difficult when the number of attributes increases, especially when some of these attributes are optional. Traditional object creation techniques often lead to code that is hard to read, maintain, and extend.

These challenges result in several common issues, which are discussed below.

2.1.1 Telescoping constructors:

Telescoping constructors occur when a class defines multiple constructors with different numbers of parameters to handle various initialization scenarios.

As the number of parameters increases, the number of constructors grows significantly, making the code difficult to manage and understand. This approach also reduces readability, as it becomes unclear what each parameter represents when many are passed in a single constructor call.

For example, passing multiple values (or nulls) into a constructor can make the code confusing and error-prone.

2.1.2 Too many parameters:

When a constructor requires a large number of parameters, it becomes difficult for developers to remember the correct order and meaning of each parameter.

This increases the likelihood of mistakes, such as passing incorrect values or mixing up parameter positions. It also reduces code readability, making it harder for others to understand the purpose of each argument.

Additionally, handling optional parameters becomes inconvenient, as developers may be

forced to pass unnecessary or placeholder values.

2.1.3 Inconsistent object state:

Using traditional approaches, such as creating an object and then setting its properties through multiple setter methods, can lead to inconsistent or incomplete object states.
An object may be used before all required fields are properly initialized, which can cause unexpected behavior or runtime errors. This lack of control over the object's state reduces reliability and makes debugging more difficult.

2.2 Why traditional approaches fail:

Traditional object creation methods, such as overloaded constructors and setter-based initialization, fail to provide a clean and reliable solution for building complex objects. Overloaded constructors lead to code duplication and poor scalability, while setter methods allow objects to exist in incomplete or invalid states. Both approaches reduce code clarity and increase the risk of errors.
As a result, these methods do not effectively handle the complexity of modern software systems, where flexibility, readability, and maintainability are essential. This creates the need for a more structured approach, such as the Builder Design Pattern, which addresses these limitations.

2.3 Possible design patterns to solve these issues:

2.3.1 Builder Design Pattern

The Builder pattern constructs objects step by step, allowing each attribute to be set individually. It provides high flexibility and ensures that the final object is created in a valid state.
It is particularly suitable for creating complex objects with many optional parameters, such as customizable game characters.

2.3.2 Abstract Factory Design Pattern

The Abstract Factory pattern is used to create families of related objects without specifying their concrete classes. In the context of game development, it could be used to create predefined sets of characters or items, such as fantasy or sci-fi themes.

However, it focuses on creating groups of related objects rather than constructing a single complex object step by step.

Pattern Selection & Justification

3.1 Why Builder Design Pattern is Needed:

In modern software development, objects often contain a large number of attributes, some of which are optional. Using traditional constructors to initialize such objects can lead to complex and unreadable code, especially when many parameters are involved.
The Builder Design Pattern is needed to simplify the creation of such complex objects. It provides a clear and flexible way to construct objects step by step, allowing developers to include only the necessary attributes without overcomplicating the code.
This pattern is particularly useful when:

- **An object has many optional parameters**
- **The construction process involves multiple steps**
- **Readability and maintainability are important**

By using the Builder pattern, developers can avoid common issues such as telescoping constructors and inconsistent object states, while making the code easier to understand and extend.

3.2 Real-World Analogy:

A common real-world example of the Builder pattern is ordering a meal at a restaurant. When ordering a burger, the customer can choose different components such as:

- **Bread type**
- **Meat or chicken**
- **Cheese**
- **Vegetables**
- **Sauces**

The final product (the burger) is built step by step based on the customer's choices. Not all components are required, and each customer can customize their order differently.

Similarly, in software, the Builder pattern allows an object to be constructed step by step, where each part is added as needed until the final object is complete.

3.3 When to Use the Builder Pattern:

The Builder Design Pattern should be used in the following situations:

- **When an object has many parameters, especially optional ones**
- **When creating an object requires multiple steps or configurations**
- **When you want to improve code readability and avoid complex constructors**
- **When you need to ensure that an object is always created in a valid and complete state**
- **When the same construction process can produce different representations of an object**

Using the Builder pattern in these scenarios leads to cleaner, more maintainable, and less error-prone code.

Case Study: Game Character Creation and UML Diagrams

4.1 Problem in Game Character Creation:

In many games, creating a character involves multiple attributes such as name, type, health, weapon, armor, and skills. Some of these attributes are required, while others are optional or vary depending on the type of character.

Using traditional object creation methods can lead to several issues. For example, if constructors are used, the class may require many parameters, making it difficult to read and use. Developers may also need to create multiple overloaded constructors to handle different combinations of attributes.

Additionally, using setter methods to assign values after object creation can result in incomplete or inconsistent objects. A character might be created without essential attributes like health or weapon, which can cause errors during gameplay.

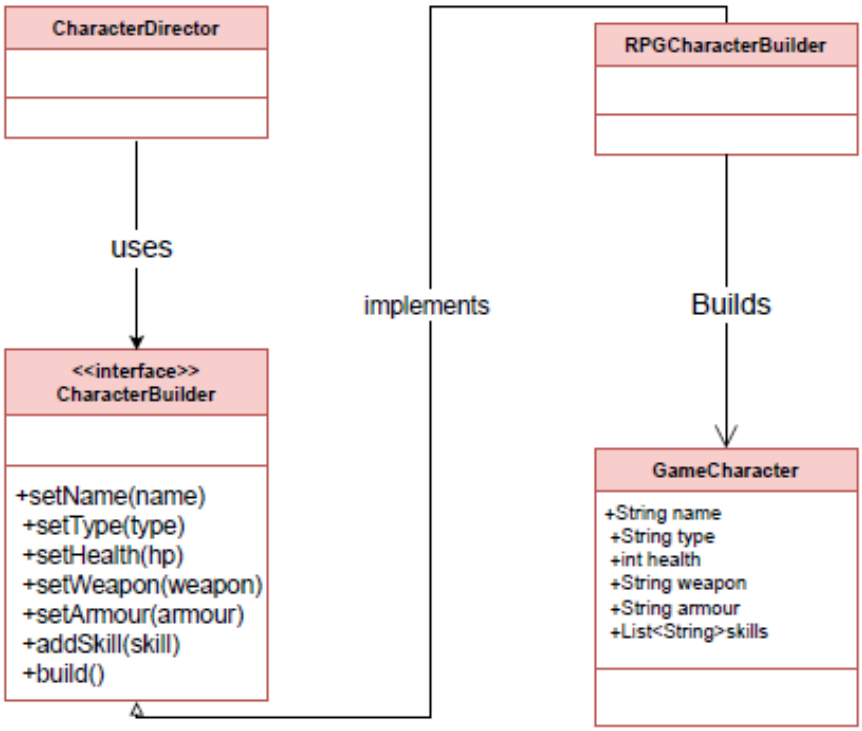
As the complexity of the character increases, managing these attributes becomes more difficult, leading to code that is hard to maintain and extend.

4.2 Applying Builder Pattern as a Solution:

The Builder Design Pattern provides a structured solution to this problem by allowing the game character to be created step by step.

Instead of passing all attributes at once, the Builder pattern enables the construction of a character by setting each property individually through a builder object. This ensures that only relevant attributes are included, and the final object is created in a complete and valid state.

For example, a character can be built by specifying its name, type, health, weapon, armor, and skills incrementally. The construction process is controlled, making the code more readable and flexible.



The system consists of:

1. A GameCharacter class representing the final product

2. A CharacterBuilder interface defining the building steps

3. A Concrete Builder (RPGCharacterBuilder) implementing these steps

4. A Director (CharacterDirector) managing the construction process, This separation allows different types of characters to be created using the same building process while maintaining clarity & consistency.

Applying the Builder pattern to game character creation provides several advantages:

- **Improves readability by constructing characters step by step**
- **Allows flexible customization of characters**
- **Ensures that characters are always created in a valid state**
- **Reduces complexity by avoiding multiple constructors**
- **Makes the system easier to extend with new character types or attributes**

Pattern Implementation

5.1 Code without Builder:

In traditional OOP design, the creation of complex objects, such as a game character, is typically handled using constructors that require many parameters:

```java
//without the builder patternn
package code1;

import java.util.List;

public class GameCharacter {
    private final String name;
    private final String type;
    private final int health;
    private final String weapon;
    private final String armour;
    private final List<String> skills;

    public GameCharacter(String name, String type, int health,String weapon, String armour, List<String> skills) {


        if (name == null || type == null) {
            throw new IllegalArgumentException("Name and type are required");
        }

        this.name = name;
        this.type = type;
        this.health = health;
        this.weapon = weapon;
        this.armour = armour;
        this.skills = List.copyOf(skills);
    }

    public String getName() { return name; }
    public String getType() { return type; }
    public int getHealth() { return health; }
    public String getWeapon() { return weapon; }
    public String getArmour() { return armour; }
    public List<String> getSkills() { return skills; }
}
```

```java
//without the builder pattern
package code1;
 import java.util.Arrays;

public class Code1 {
    public static void main(String[] args) {

        GameCharacter character = new GameCharacter(
                "Ali",
                "Warrior",
                150,
                "Sword",
                "Heavy",
                Arrays.asList("Slash", "Block")
        );

        System.out.println(character.getName());
    }
}
```

As more parameters are introduced, the constructor becomes harder to read and understand. Developers must remember the exact order and meaning of each parameter, which significantly increases the likelihood of errors. Furthermore, optional attributes must still be provided, often in the form of null or default values, which reduces code clarity and expressiveness.

This method lacks scalability and does not effectively support the construction of complex and flexible objects, making it unsuitable for real-world applications such as game character creation.

5.2 Code with Builder:

To overcome this risk, the Builder Design Pattern provides a more structured and flexible approach to object construction:

```java
package codewithbuilder;
interface CharacterBuilder {
    CharacterBuilder setName(String name);
    CharacterBuilder setType(String type);
    CharacterBuilder setHealth(int hp);
    CharacterBuilder setWeapon(String weapon);
    CharacterBuilder setArmour(String armour);
    CharacterBuilder addSkill(String skill);
    GameCharacter build();
}
```

```java
package codewithbuilder;

import java.util.List;

class GameCharacter {
    private final String name;
    private final String type;
    private final int health;
    private final String weapon;
    private final String armour;
    private final List<String> skills;

    public GameCharacter(String name, String type, int health,String weapon, String armour, List<String> skills) {


        this.name = name;
        this.type = type;
        this.health = health;
        this.weapon = weapon;
        this.armour = armour;
        this.skills = List.copyOf(skills);
    }

    public String getName() { return name; }
    public String getType() { return type; }
}
```

```java
package codewithbuilder;
import java.util.ArrayList;
import java.util.List;

class RPGCharacterBuilder implements CharacterBuilder {

    private String name;
    private String type;
    private int health = 100;
    private String weapon;
    private String armour;
    private List<String> skills = new ArrayList<>();

    @Override
    public CharacterBuilder setName(String name) {
        this.name = name;
        return this;
    }

    @Override
    public CharacterBuilder setType(String type) {
        this.type = type;
        return this;
    }

    @Override
    public CharacterBuilder setHealth(int hp) {
        this.health = hp;
        return this;
    }

    @Override
    public CharacterBuilder setWeapon(String weapon) {
        this.weapon = weapon;
        return this;
```

```java
    }

    @Override
    public CharacterBuilder setWeapon(String weapon) {
        this.weapon = weapon;
        return this;
    }

    @Override
    public CharacterBuilder setArmour(String armour) {
        this.armour = armour;
        return this;
    }

    @Override
    public CharacterBuilder addSkill(String skill) {
        this.skills.add(skill);
        return this;
    }

    @Override
    public GameCharacter build() {
        if (name == null || type == null) {
            throw new IllegalStateException("Name and type are required");
        }

        return new GameCharacter(name, type, health, weapon, armour, skills);
    }
}
```

```java
package codewithbuilder;

class CharacterDirector {

    public GameCharacter createWarrior() {
        return new RPGCharacterBuilder()
                .setName("Default Warrior")
                .setType("Melee")
                .setHealth(150)
                .setWeapon("Sword")
                .setArmour("Heavy")
                .addSkill("Slash")
                .build();
    }
}
```

```java
package codewithbuilder;

public class Codewithbuilder {
    public static void main(String[] args) {


//   without the director

        GameCharacter character = new RPGCharacterBuilder()
                .setName("Ali")
                .setType("Warrior")
                .setHealth(150)
                .setWeapon("Sword")
                .addSkill("Slash")
                .build();

        System.out.println(character.getName());


//   with the  director
CharacterDirector director = new CharacterDirector();
        GameCharacter warrior = director.createWarrior();

        System.out.println(warrior.getName());
```

In this approach, the construction process is clearly separated from the final representation of the object. The Builder class allows attributes to be set incrementally through well-defined and descriptive methods.

This step-by-step construction process enhances readability and provides greater control over object creation. It also allows developers to include only the necessary attributes while omitting optional ones without affecting code clarity.

5.3 Key differences:

The primary differences between the traditional approach and the Builder pattern lie in readability, flexibility, and scalability.

In the traditional approach, object creation depends heavily on constructors, which become increasingly complex as more parameters are added. This leads to reduced clarity and a higher probability of errors.

The Builder pattern enables step-by-step object construction, where each attribute is explicitly defined through meaningful method calls. This results in a clearer and more maintainable structure.

Additionally, the Builder pattern enhances flexibility by allowing optional attributes to be omitted without impacting the overall construction process, thereby making it highly

suitable for complex and scalable systems.

Advantages of Builder Pattern

Advantages:

The Builder Design Pattern offers several key advantages that make it highly suitable for complex object creation.

1. It significantly improves code readability by breaking down the construction process into clear and logical steps. Each attribute is explicitly defined, making the code self-explanatory.

2. It provides strong support for optional parameters, allowing developers to construct objects with only the required attributes while omitting unnecessary ones.

3. The pattern reduces the likelihood of errors by eliminating the need to remember parameter order, which is a common issue in constructor-based approaches.

4. It enhances maintainability and scalability, as new attributes can be added with minimal changes to existing code, making the system more adaptable to future requirements.

Disadvantages of Builder Pattern

Disadvantages:

Despite its advantages, the Builder pattern introduces certain trade-offs that should be considered:

1. The increased complexity in the code structure requires additional classes or inner classes to implement the builder.

2. Results in more verbose code compared to simple constructors, which may not be ideal for small or simple objects.

3. In scenarios where objects are relatively simple, the use of the Builder pattern may be unnecessary and could introduce additional overhead without significant benefits.

Conclusion

8.1 Summary of the pattern:

The Builder Design Pattern is a creational design pattern that focuses on constructing complex objects step by step.

It allows developers to create objects in a clear and flexible manner while ensuring consistency and validity. By separating the construction logic from the final representation, the pattern enables different variations of the same object to be created using the same process.

This structured approach makes the Builder pattern highly effective for developing scalable and maintainable systems.

8.2 Why it was chosen:

The Builder Design Pattern was chosen for the game character creation system due to the inherent complexity of the object.

Game characters typically consist of multiple attributes, some of which are optional or vary depending on the character type. Using traditional constructors in such cases leads to reduced clarity and increased risk of errors.

By applying the Builder pattern, characters can be constructed step by step, allowing developers to include only the necessary attributes while maintaining a clear and readable structure.

This approach not only simplifies the creation process but also ensures that all objects are created in a valid and consistent state.

Additionally, the pattern supports future extensibility, making it easier to introduce new character types or attributes without requiring significant modifications to the existing system.

References

Books

1. Design Patterns: Elements of Reusable Object-Oriented Software

   Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994).

   Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Websites

1. Refactoring Guru

   Builder Design Pattern.

   https://refactoring.guru/design-patterns/builder

2. GeeksforGeeks

   Builder Design Pattern in Java.

   https://www.geeksforgeeks.org/builder-design-pattern/